



PROJETO E ANÁLISE DE ALGORITMOS

AULA 11 – ALGORITMOS GULOSOS

11.1 ALGORITMOS GULOSOS

- Normalmente, os algoritmos para problemas de otimização passam por uma sequência de etapas, e cada etapa tem um conjunto de escolhas. Para muitos problemas de otimização é um **exagero utilizar programação dinâmica** para determinar as melhores escolhas, pois algoritmos mais simples e mais eficientes servirão.
- Um **algoritmo guloso** sempre faz a escolha que parece ser a melhor no momento em questão. Isto é, faz uma escolha localmente ótima, na esperança de que essa escolha leve a uma solução globalmente ótima.

11.1 ALGORITMOS GULOSOS

- Normalmente, os algoritmos para problemas de otimização passam por uma sequência de etapas, e cada etapa tem um conjunto de escolhas. Para muitos problemas de otimização é um **exagero utilizar programação dinâmica** para determinar as melhores escolhas, pois algoritmos mais simples e mais eficientes servirão.
- Um **algoritmo guloso** sempre faz a escolha que parece ser a melhor no momento em questão. Isto é, faz uma escolha localmente ótima, na esperança de que essa escolha leve a uma solução globalmente ótima.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Consideraremos o problema de programar várias atividades concorrentes que requerem o uso exclusivo de um recurso comum, com o objetivo de selecionar um conjunto de tamanho máximo de atividades mutuamente compatíveis.
- Imagine que você esteja encarregado de reservar uma sala de conferências. Você tem um conjunto $S = \{a_1, a_2, \dots, a_n\}$ de n atividades propostas que desejam usar a sala de conferências, e a sala só pode ser utilizada uma única atividade por vez.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Cada atividade a_i tem um tempo de início s_i e um tempo de término f_i , em que $0 \leq s_i < f_i < \infty$.
- Se selecionada, a atividade a_i ocorre durante o intervalo de tempo meio aberto $[s_i, f_i)$.
- As atividades a_i e a_j são compatíveis se os intervalos $[s_i, f_i)$ e $[s_j, f_j)$ não se sobrepõem. Isto é, a_i e a_j são compatíveis se $s_i \geq f_j$ ou $s_j \geq f_i$.
- No problema de seleção de atividades, queremos selecionar um **subconjunto de tamanho máximo de atividades mutuamente compatíveis**.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Supomos que as atividades estão organizadas em ordem monotonicamente crescente de tempo de término:

$$f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n$$

- Por exemplo, considere o conjunto de atividades na tabela a seguir. O subconjunto $\{a_3, a_9, a_{11}\}$ consiste em atividades mutuamente compatíveis. Porém, não é um subconjunto máximo, já que o subconjunto $\{a_1, a_4, a_8, a_{11}\}$ é maior. De fato, $\{a_1, a_4, a_8, a_{11}\}$ é um dos subconjuntos maiores de atividades mutuamente compatíveis; outro subconjunto maior é $\{a_2, a_4, a_9, a_{11}\}$.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Devemos determinar que o problema possui **subestrutura ótima**.
- Vamos indicar por S_{ij} o conjunto de atividades que começam após o término da atividade a_i e terminam antes do início da atividade a_j .
- Suponha que queremos determinar um conjunto máximo de atividades mutuamente compatíveis em S_{ij} , e suponha ainda mais que tal subconjunto máximo é A_{ij} , que inclui alguma atividade a_k .

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Incluindo a_k em uma solução ótima, ficamos com dois subproblemas em mãos: determinar atividades mutuamente compatíveis no conjunto S_{ik} (atividades que começam após o término da atividade a_i e terminam antes do início da atividade a_k) e determinar atividades mutuamente compatíveis no conjunto S_{kj} (atividades que começam após o término da atividade a_k e terminam antes do início da atividade a_j).

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Sejam $A_{ik} = A_{ij} \cap S_{ik}$ e $A_{kj} = A_{ij} \cap S_{kj}$, de modo que A_{ik} contém as atividades em A_{ij} que terminam antes do início de a_k e A_{kj} contém as atividades em A_{ij} que começam após o término de a_k .
- Assim, temos $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ e, portanto, o conjunto de tamanho máximo A_{ij} de atividades mutuamente compatíveis em S_{ij} consiste em $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$ atividades.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- A solução ótima A_{ij} deve incluir também soluções ótimas para os dois subproblemas de S_{ik} e S_{kj} .
- Se pudéssemos determinar um conjunto A'_{kj} de atividades mutuamente compatíveis em S_{kj} com $|A'_{kj}| > |A_{kj}|$, poderíamos usar A'_{kj} , em vez de A_{kj} , em uma solução do subproblema para S_{ij} .
- Poderíamos ter construído um conjunto de $|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$ atividades mutuamente compatíveis, o que contradiz a suposição de que A_{ij} seja uma solução ótima. Um argumento simétrico se aplica às atividades em S_{ik} .

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Esse modo de caracterizar subestrutura ótima sugere que poderíamos resolver o problema de seleção de atividades por programação dinâmica. Vamos indicar o tamanho de uma solução ótima para o conjunto S_{ij} por $c[i, j]$.
- Então, a abordagem da programação dinâmica produz a recorrência:

$$c[i, j] = c[i, k] + c[k, j] + 1$$

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- É claro que, se não soubermos que uma solução ótima para o conjunto S_{ij} inclui a atividade a_k , teremos de examinar todas as atividades em S_{ij} para determinarmos qual delas escolher, de modo que:

$$c[i, j] = \begin{cases} 0 & \text{se } S_{ij} = \emptyset \\ \max\{c[i, k] + c[k, j] + 1 : a_k \in S_{ij}\} & \text{se } S_{ij} \neq \emptyset \end{cases}$$

- Então, poderemos desenvolver um algoritmo recursivo e memoizá-lo, ou trabalhar de baixo para cima e preencher entradas de tabela durante o processo.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Porém, estaríamos ignorando outra característica importante do problema de seleção de atividades que podemos usar e que seria muito vantajoso.
- E se pudéssemos escolher uma atividade para acrescentar à nossa solução ótima **sem termos de resolver primeiro todos os subproblemas?**
- Isso nos pouparia de ter de considerar **todas as escolhas** inerentes à recorrência anterior.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Na verdade, no caso do problema de seleção de atividades, precisamos considerar somente uma escolha: a escolha gulosa.
- A intuição sugere que deveríamos escolher uma atividade que deixasse o recurso disponível para o maior número possível de outras atividades. Agora, entre as atividades que acabamos escolhendo, uma deve ser a primeira a terminar.
- Portanto, nossa intuição nos diz para escolher a atividade em S que tenha o tempo de término mais próximo, já que isso deixará o recurso disponível para o maior número possível de atividades que ocorram depois dessa.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Se mais de uma atividade em S tiver o mesmo tempo mais próximo de término, poderemos escolher qualquer uma delas.
- Em outras palavras, visto que as atividades são classificadas em ordem monotônica crescente por tempo de término, a escolha gulosa é a atividade a_1 .
- Escolher a primeira atividade a terminar não é o único modo de fazer uma escolha gulosa para esse problema.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Se fizermos a escolha gulosa, restará somente um subproblema para resolver: determinar atividades que comecem após o término de a_1 .
- Por que não temos de considerar atividades que terminam antes de a_1 começar?
- Temos que $s_1 < f_1$, e f_1 é o tempo mais próximo de término de qualquer atividade; portanto, nenhuma atividade pode ter um tempo de término menor ou igual a s_1 . Assim, todas as atividades que são compatíveis com a atividade a_1 devem começar depois que a_1 terminar.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Além disso, já estabelecemos que o problema de seleção de atividades exibe subestrutura ótima. Seja $S_k = \{a_i \in S : s_i \geq f_k\}$ o conjunto de atividades que começam após o término de a_k .
- Se fizermos a escolha gulosa da atividade a_1 , S_1 permanecerá como o **único problema a resolver**.
- A subestrutura ótima nos diz que, se a_1 pertencer a uma solução ótima, então uma solução ótima para o problema original consistirá na atividade a_1 e todas as atividades em uma solução ótima para o subproblema S_1 .

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Teorema: considere um subproblema qualquer não vazio S_k , e seja a_m uma atividade em S_k com o tempo mais próximo de término. Então, a_m estará incluída em algum subconjunto de tamanho máximo de atividades mutuamente compatíveis de S_k .
- Ainda que pudéssemos resolver o problema da seleção de atividades com **programação dinâmica**, o teorema diz que não precisamos fazê-lo. Em vez disso, podemos escolher repetidamente a atividade que termina primeiro, manter somente as atividades compatíveis com essa atividade e repetir o processo até não restar nenhuma atividade.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Além disso, como sempre escolhemos a atividade que tem o tempo de término mais próximo, os tempos de término das atividades que escolhermos deve crescer estritamente. Podemos considerar cada atividade apenas uma vez no total, em ordem monotonicamente crescente de tempos de término.
- Um algoritmo para resolver o problema de seleção de atividades não precisa funcionar de baixo para cima, como um algoritmo de programação dinâmica baseado em tabela.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Em vez disso, pode trabalhar de cima para baixo, escolhendo uma atividade para colocar na solução ótima e, então, resolvendo o problema de escolher atividades entre as que sejam compatíveis com as já escolhidas.
- Normalmente, algoritmos gulosos têm o seguinte projeto de cima para baixo: **faça uma escolha e resolva um subproblema, em vez da técnica de baixo para cima**, que resolve subproblemas antes de fazer uma escolha.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- O procedimento SELECIONA-ATIVIDADE-RECURSIVO toma os tempos de início e término das atividades, representados como vetores s e f , o índice k , que define o subproblema S_k que ele deve resolver, e o tamanho n do problema original.
- Ele retorna um conjunto de tamanho máximo de atividades mutuamente compatíveis em S_k .
- Consideramos que as n atividades de entrada já estão organizadas em ordem monotonicamente crescente de tempos de término.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

SELECIONA-ATIVIDADE-RECURSIVO(s , f , k , n)

1 $m = k + 1$

2 while $m \leq n$ e $s[m] < f[k]$ // acha a primeira atividade em S_k a terminar

3 $m = m + 1$

4 if $m \leq n$

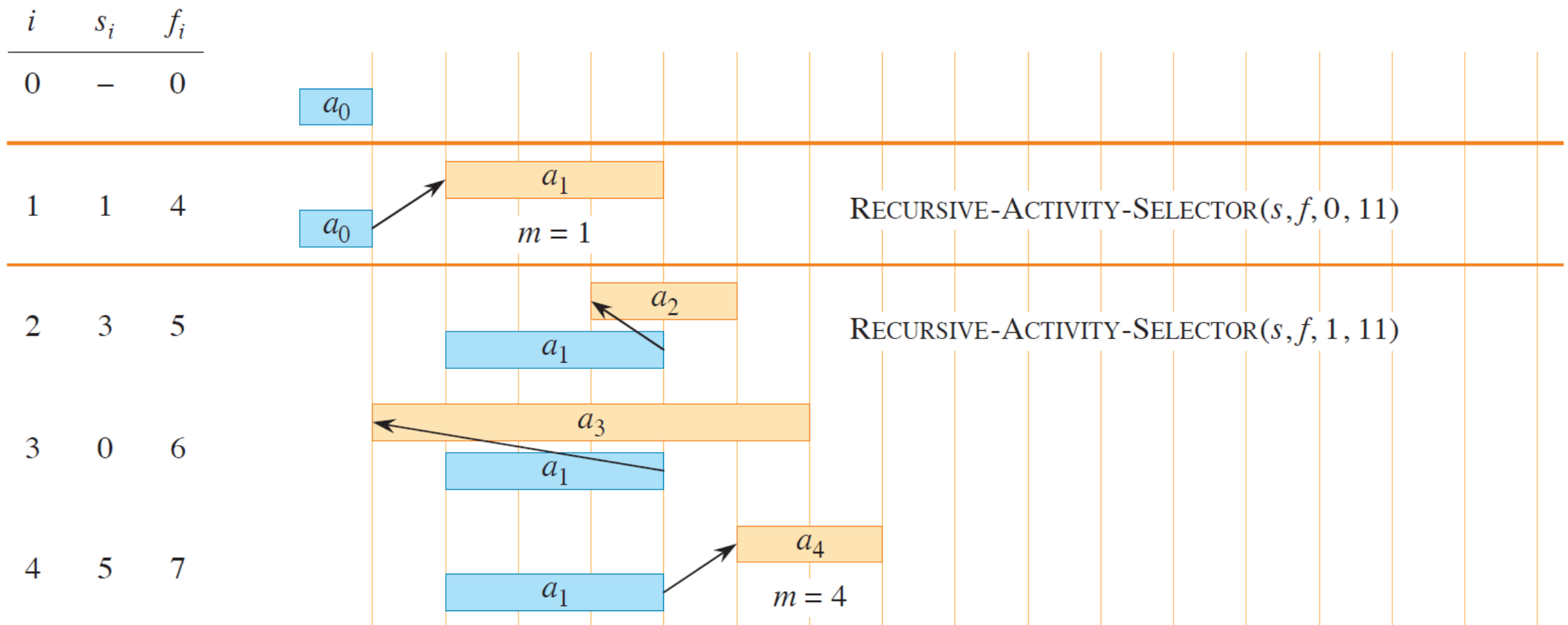
5 return $\{a_m\} \cup \text{SELECIONA-ATIVIDADE-RECURSIVO}(s, f, m, n)$

6 else return $\emptyset$

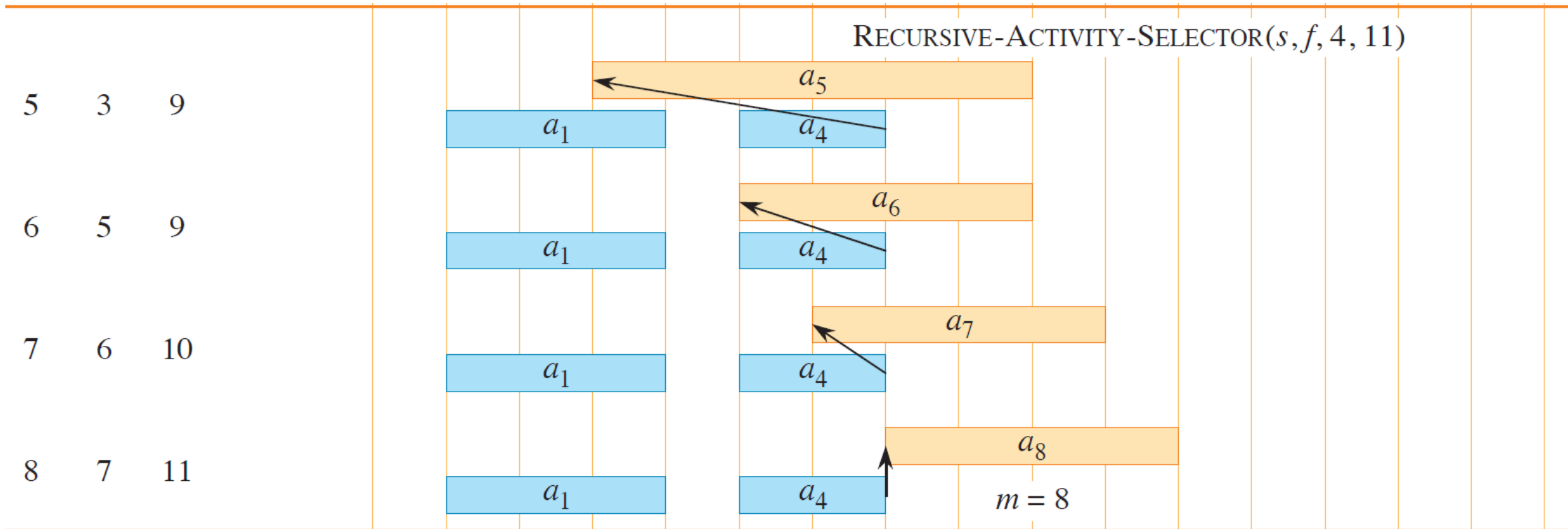
11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- Para começar, acrescentamos a atividade fictícia a_0 com $f_0 = 0$, de modo que o subproblema S_0 é o conjunto inteiro de atividades S .
- A chamada inicial, que resolve o problema inteiro, é SELECIONA-ATIVIDADE-RECURSIVO($s, f, 0, n$).
- Considerando que as atividades já foram ordenadas por tempos de término, o tempo de execução da chamada SELECIONA-ATIVIDADE-RECURSIVO($s, f, 0, n$) é $\Theta(n)$. Para entender o motivo, observe que, em todas as chamadas recursivas, cada atividade é examinada exatamente uma vez no teste do laço while da linha 2. Em particular, a atividade a_i é examinada na última chamada feita na qual $k < i$.

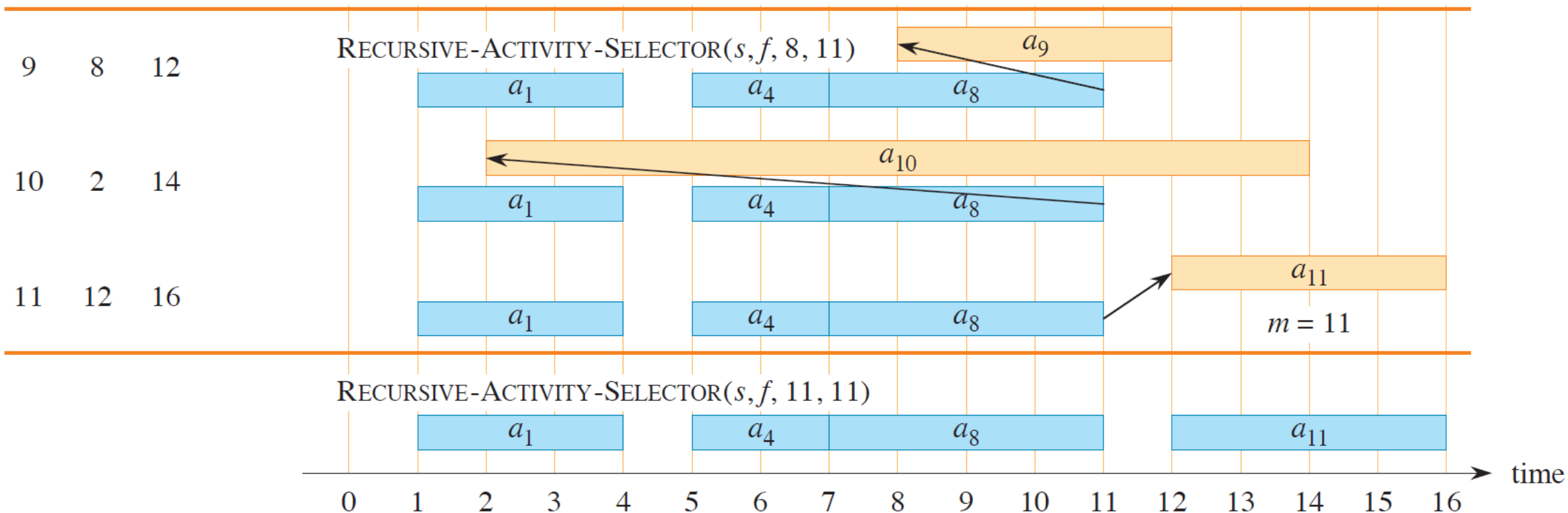
11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES



11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES



11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES



11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- O procedimento SELECIONA-ATIVIDADE-GULOSO é uma versão iterativa do procedimento SELECIONA-ATIVIDADE-RECURSIVO.
- Ele também considera que as atividades de entrada estão organizadas em ordem monotonicamente crescente de tempos de término.
- O procedimento reúne atividades selecionadas em um conjunto A e retorna esse conjunto quando termina.

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

SELECIONA-ATIVIDADE-GULOSO(s , f , n)

1 $A = \{a_1\}$

2 $k = 1$

3 for $m = 2$ até n

4 if $s[m] \geq f[k]$ // a_m está em S_k ?

5 $A = A \cup \{a_m\}$ // sim, então a escolhe

6 $k = m$ // e continua a partir daí

7 return A

11.2 PROBLEMA DE SELEÇÃO DE ATIVIDADES

- O conjunto A retornado pela chamada SELECIONA-ATIVIDADE-GULOSO(s , f) é exatamente o conjunto retornado pela chamada SELECIONA-ATIVIDADE-RECURSIVO(s , f , 0, n).
- Como na versão recursiva, o procedimento SELECIONA-ATIVIDADE-GULOSO programa um conjunto de n atividades no tempo $\Theta(n)$, considerando que as atividades já estavam ordenadas inicialmente por seus tempos de término.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- Um algoritmo guloso obtém uma **solução ótima** de problema fazendo uma sequência de escolhas.
- Para cada ponto de decisão, o algoritmo escolhe a opção que **parece melhor no momento**.
- Essa **estratégia heurística nem sempre produz uma solução ótima**, mas, como vimos no problema de seleção de atividades, **algumas vezes, funciona**.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- Para o problema da seleção de atividades Seguimos estas etapas:
 1. Determinar a **subestrutura ótima** do problema.
 2. Desenvolver uma **solução recursiva**.
 3. Mostrar que, **se fizermos a escolha gulosa, restará somente um subproblema**.
 4. **Provar que é sempre seguro fazer a escolha gulosa.** (As etapas 3 e 4 podem ocorrer em qualquer ordem.)
 5. Desenvolver um **algoritmo recursivo** que implemente a estratégia gulosa.
 6. **Converter o algoritmo recursivo em um algoritmo iterativo.**

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- Como alternativa, poderíamos ter conformado nossa subestrutura ótima tendo em mente uma **escolha gulosa**, de modo que a escolha deixasse apenas um problema para resolver.
- No problema de seleção de atividades, poderíamos ter começado descartando o segundo índice e definindo subproblemas da forma S_k . Então, poderíamos ter provado que uma escolha gulosa (a primeira atividade a_m a terminar em S_k), combinada com uma solução ótima para o conjunto restante S_m de atividades compatíveis, produz uma solução ótima para S_k .

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- De modo mais geral, projetamos algoritmos gulosos de acordo com esta sequência de etapas:
 1. **Expressar o problema de otimização** como problema no qual fazemos uma escolha e ficamos com um **único subproblema** para resolver.
 2. Provar que sempre existe uma **solução ótima** para o problema original que usa a escolha gulosa, de modo que a escolha gulosa é **sempre segura**.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- De modo mais geral, projetamos algoritmos gulosos de acordo com esta sequência de etapas:
 3. **Demonstrar subestrutura ótima** mostrando que, tendo feito a escolha gulosa, o que resta é um subproblema com a seguinte propriedade: se combinarmos uma solução ótima para o subproblema com a escolha gulosa que fizemos, chegamos a uma solução ótima para o problema original.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- **Propriedade de escolha gulosa:**
 - Podemos montar uma solução globalmente ótima fazendo escolhas (gulosas) locais ótimas. Em outras palavras, quando estamos considerando qual escolha fazer, escolhemos a que **parece melhor** para o problema em questão, sem considerar resultados de subproblemas.
 - É nesse ponto que os algoritmos gulosos são **diferentes** da programação dinâmica. Na programação dinâmica, fazemos uma escolha em cada etapa, mas, normalmente, a escolha depende das soluções para **subproblemas**.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

- **Propriedade de escolha gulosa:**
 - Diferentemente da programação dinâmica, que resolve os subproblemas antes de fazer a primeira escolha, **um algoritmo guloso faz sua primeira escolha antes de resolver qualquer subproblema**. Um algoritmo de programação dinâmica age de baixo para cima, ao passo que uma estratégia gulosa, em geral, age de cima para baixo, fazendo uma escolha gulosa após outra, reduzindo cada instância do problema dado a uma instância menor.
 - É claro que temos de provar que uma escolha gulosa em cada etapa produz uma **solução globalmente ótima**.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

■ Subestrutura ótima

- Um problema exibe subestrutura ótima se uma solução ótima para o problema contiver soluções ótimas para subproblemas. Essa propriedade é um componente fundamental para avaliar se a programação dinâmica se aplica, e também é essencial para algoritmos gulosos. No problema das atividades, se uma solução ótima para o subproblema S_{ij} incluir uma atividade a_k , então ela também deve conter soluções ótimas para os subproblemas S_{ik} e S_{kj} .

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

■ Subestrutura ótima

- Dada essa subestrutura ótima, demonstramos que, se soubéssemos qual atividade usar como a_k , poderíamos construir uma solução ótima para S_{ij} selecionando a_k juntamente com todas as atividades em soluções ótimas para os subproblemas S_{ik} e S_{kj} . Com base nessa observação de subestrutura ótima, pudemos criar a recorrência que descreve o valor de uma solução ótima.

11.3 ELEMENTOS DA ESTRATÉGIA GULOSA

■ Subestrutura ótima

- Normalmente, usamos uma abordagem mais direta com relação à subestrutura ótima quando a aplicamos em algoritmos gulosos. Podemos nos permitir o luxo de supor que chegamos a um subproblema por termos feito a escolha gulosa no problema original. Na realidade, basta que demonstremos que uma solução ótima para o subproblema, combinada com a escolha gulosa já feita, produz uma solução ótima para o problema original.

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- Como as estratégias gulosa e de programação dinâmica exploram a subestrutura ótima, você poderia ser tentado a gerar uma solução de programação dinâmica para um problema quando uma solução gulosa seria suficiente ou, ao contrário, achar erroneamente que uma solução gulosa funciona quando, na verdade, seria preciso uma solução de programação dinâmica.
- Para ilustrarmos as sutilezas entre as duas técnicas, vamos investigar duas variantes de um problema clássico de otimização.

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- Considere o **problema binário da mochila**. Um ladrão que assalta uma loja deseja levar a carga mais valiosa que possa ser transportada em uma mochila capaz de carregar no máximo W quilos. O ladrão pode escolher levar um subconjunto de n itens na loja. O i -ésimo item vale v_i reais e pesa ω_i quilos, em que v_i e ω_i são inteiros.
- Que itens ele deve levar?
 - Esse problema da mochila chama-se binário porque, para cada item, o ladrão tem de decidir se o carrega ou se o deixa para trás; ele não pode levar uma quantidade fracionária de um item nem um item mais de uma vez.

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- No problema fracionário da mochila, a configuração é a mesma, mas o ladrão pode levar frações de itens, em vez de ter de fazer uma escolha binária para cada item.
- Para perceber melhor a diferença, imagine que um item no problema binário da mochila seja como uma barra de ouro, enquanto um item no problema da mochila fracionário seria semelhante a ouro em pó.

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- Ambos os problemas da mochila exibem a propriedade de subestrutura ótima.
- No caso do problema binário, se a carga mais valiosa, que pesa no máximo W quilos, inclui o item j , então a carga restante deverá ser a carga mais valiosa que pese no máximo $W - \omega_j$ que o ladrão pode levar dos $n - 1$ itens originais, excluindo o item j .
- No caso do problema fracionário, se removermos um peso ω de um item j da carga ótima, a carga restante deverá ser a carga mais valiosa que pese no máximo $W - \omega$ quilos que o ladrão pode levar dos $n - 1$ itens originais mais $\omega_j - \omega$ quilos do item j .

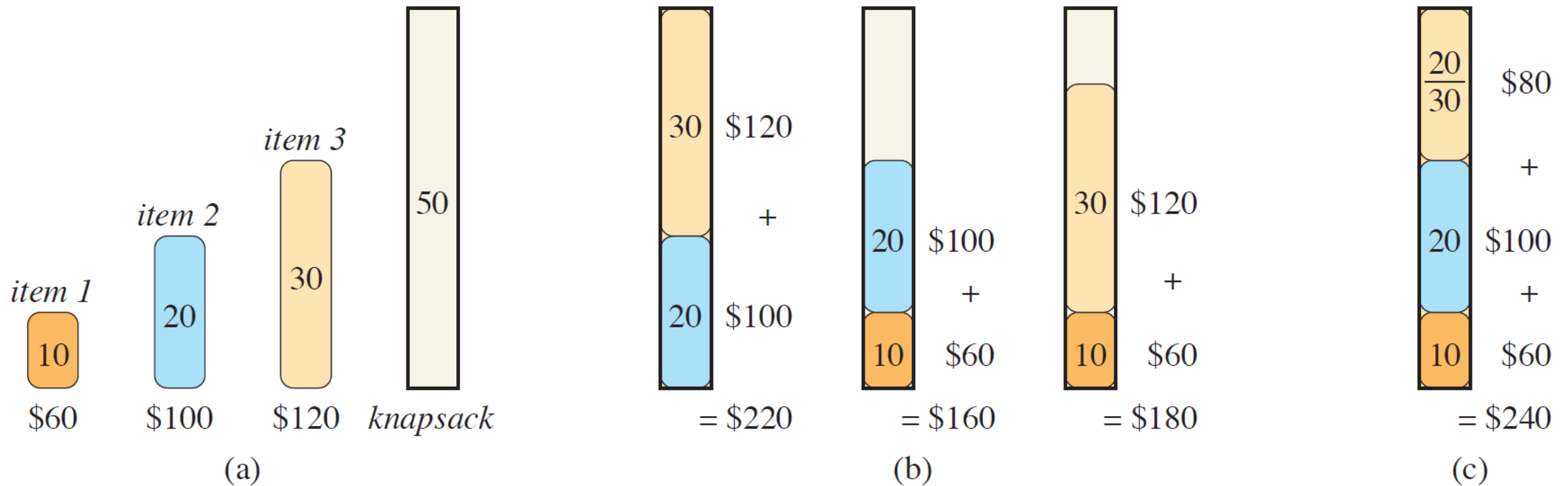
11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- Embora os problemas sejam **semelhantes**, podemos resolver o problema fracionário da mochila por uma estratégia gulosa, mas não podemos usar essa mesma estratégia para resolver o problema binário.
- Para resolvermos o problema fracionário, primeiro calculamos o valor por quilo v_i / ω_i para cada item. Obedecendo a uma estratégia gulosa, o ladrão começa levando o máximo possível do item que tenha o maior valor por quilo.

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- Se o suprimento desse item se esgotar e o ladrão ainda puder carregar mais, levará o máximo possível do próximo item que tenha o maior valor por quilo e assim por diante até alcançar seu peso limite W . Portanto, ordenando os itens conforme seu valor por quilo, o algoritmo guloso é executado no tempo $O(n \lg n)$.
- Para ver que essa estratégia gulosa não funciona para o problema binário da mochila, considere a instância do problema ilustrada na Figura a seguir. Esse exemplo tem três itens e uma mochila que pode conter 50 quilos (a).

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA



11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- A estratégia gulosa levaria o item 1 primeiro. Porém, como podemos ver pela análise de caso em (b), a solução ótima leva os itens 2 e 3 e deixa o item 1 para trás. As duas soluções possíveis que envolvem o item 1 são subótimas.
- A estratégia gulosa para o problema fracionário, que leva o item 1 primeiro, realmente produz uma solução ótima, como vemos em (c). Levar o item 1 não funciona no problema binário porque o ladrão não consegue encher sua mochila até a capacidade máxima, e o espaço vazio reduz o valor efetivo por quilo de sua carga.

11.4 ESTRATÉGIA GULOSA X PROGRAMAÇÃO DINÂMICA

- No problema binário, quando consideramos incluir um item na mochila, temos de comparar a solução para o subproblema que inclua tal item com a solução para o subproblema que exclua esse mesmo item, antes de podermos fazer a escolha.
- O problema formulado desse modo dá origem a muitos subproblemas sobrepostos — uma marca registrada da programação dinâmica.

11.5 EXERCÍCIO

- **Exercício:** ler a seção **15.3** (Códigos de Huffman) do Livro Algoritmos Thomas H. Cormen.
- Opcional: seção 15.4 (Caching off-line).